Enzyme Classification Using Graphs

https://github.com/smarrapu05/stat656final

1. Project Overview

Graph based protein analysis is a vital area of bioinformatics because even incredibly structurally similar proteins can have incredibly different functions. For instance myoglobin is incredibly close to hemoglobin, but has half of the functionality since it can't deposit oxygen and only holds onto it. Understanding these sorts of specific residue and structural differences helps in fields like drug discovery and synthetic biology. In specific we are looking at a subclass of proteins called enzymes. Enzymes are a specific type of protein that catalyzes reactions in order to allow for them to occur easier. Enzymes are classified into 1 of 6 top level enzyme classes that categorize them based on the type of reaction they catalyze and how they accomplish it. Our project is classification based, where we are trying to classify these protein networks into each of these Enzyme Commission (EC) numbers.

Network based statistical methods have been used for a long time to identify a host of properties, however they are limited when it comes to multiple network analysis. Methods like multiple adjacency spectral embedding require graphs to have similar node amounts and structures, however EC numbers are too broad of categories for such methods. The proteins we're looking at have many different sizes and shapes so we are limited in traditional methodology. There are two approaches to dealing with this that we will look at.

Firstly, we will attempt to use global network properties like node centrality measures, radius and sparseness across the classes as factors in traditional machine learning methods. The specific machine learning models we used were Logistic Regression, SVM, Random Forest, and XGBoost. These methods have a critical hole, in that it is unable to effectively capture local factors like specific residue clusters. To solve this issue, we also implement graphical neural networks (GNN).

GNNs have the unique advantage of being able to intake a multitude of different sized graphs in order to capture trends. We will be using two specific kinds of GNNS, a Graph Convolutional Network (GCN) and a Graph Attention Network (GAT). GCNs use the convolution operation to pass information about not only specific nodes but also their neighbors, allowing us to capture local structures in proteins. This would allow us to find things like active sites in the protein that would be highly relevant in classifying the protein. GATs are able to specify interactions on an even smaller scale using attention weights. This approach may be particularly advantageous for proteins as we are looking for structures on the specific residue level sometimes.

2. Data Description and Preprocessing

The data for this project is the ENZYMES dataset sourced from TUDatasets, which is a collection of benchmark datasets for graph classification. This dataset is composed of 600 proteins, each of which is represented by an adjacency list .Each of the 6 top level EC classes has 100 graphs each. In addition, each node in each graph has 18 node attributes. These node attributes represent the spatial position of the node (x,y,z) along with 15 different chemical properties such as hydrophobicity, polarity, and charge. In addition, this dataset has node labels which represent the secondary structure type of the residue (helices, sheets or turns). These labels were one hot encoded which led to a total of 21 features per node in each graph.

Since this data is already processed as a benchmark dataset, there were no issues with missing values or unclean formatting. It did, however, require a number of functions in order to import it into both R and python. Because of the varying protein sizes, the edge list had to be

split into each different protein graph based on the graph indicator label file along with the node attributes and labels. In R this was solved relatively quickly with using the graph labels to index the other input files (edge list, node attributes, node labels). In Python np.bincount was incredibly useful for getting the size of each graph and working from there in order to split the other input files for each graph. As a whole the data acquisition and preprocessing were rather straightforward.

In the project proposal we considered creating a separate test set apart from the ENZYMES dataset, which would be used as a training set. Unfortunately, RCSB PDB, a data bank of thousands of proteins, was too difficult to work with in order to get a significant amount of the data we want. In particular, we ran into a big problem with the fact that a 7th top level EC class was created in 2018. The ENZYMES dataset was created before this so some proteins have changed classes according to this new classification scheme, and any protein documented after 2018 would have been classified differently. For these reasons we ended up not having time to create the test dataset from the RCSB PDB and we used the ENZYMES dataset and tested the models using 6-fold cross validation.

3. Exploratory Data Analysis

Since we are trying to look at the structural differences between proteins, we started by looking at the differences between each enzyme class across a multitude of network statistics. We looked at the average node count as a measure of size, the edge count, radius (the smallest maximum distance from any vertex to the farthest node) and diameter (the largest minimum distance between any two vertices within a graph) as measures of connectivity and eigenvector centrality. Eigenvector centrality should theoretically be quite important for proteins as the inherent weighing based on the connectivity of neighbors allows for a greater idea of what residues are structurally important and a part of important spaces like active sites.

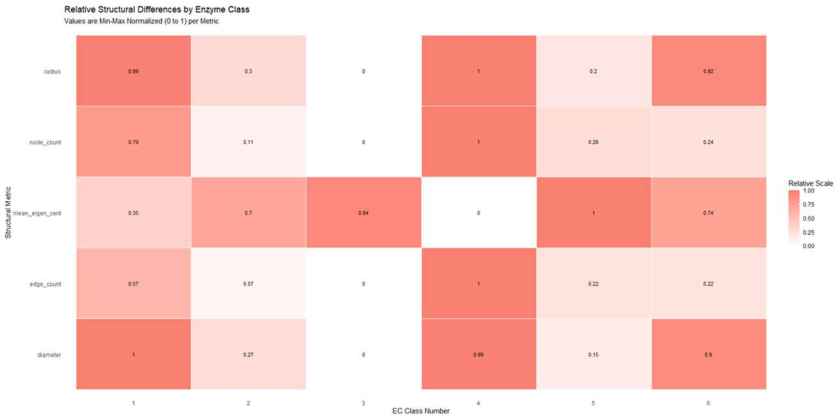

Figure 1. Heatmap of Global Network Properties

Normal centrality measures like degree centrality don't account for such things and only look at direct connections which is not particularly useful here because a lot of proteins have superfluous connections that are a consequence of actually useful structure.

Figure 1 contains a heatmap of the relative values across classes. The direct numbers and statistics are of little importance as we're trying to find comparative differences in the structure. We have a couple things of note here, Class 3 is generally smaller and more compact in order to allow for its more specific function of hydrolysis. Other classes are more general and host more possible reactions such as Class 4. There are specific group functions within class 4 proteins that have to do varying complex operations, which leads to the less central and more sparse networks. Classes 1 and 6 have relatively "spread out" network properties, with no particular stand out areas. This causes problems in the future as there will be quite a bit of misclassification between these models based on the global network structure.

In order to gain greater insight into the actual differences between classes we ran multiple ANOVA models. Across every network statistic factor individual the anova tests gave us a significant p-value (with α=0.05). This means that for each network statistic there's at least one class that is significantly different from the others. However, with 6 classes this isn't very useful on its own. In order to identify problem classes we ran Tukey's HSD across every network statistic as well. A particular problem class we found was Class 6, which was only significantly different using diameter/radius. This means that the majority of the statistics we're using won't be able to sufficiently pick class 6 apart from any other class. In addition we also found that class 2 had problems being differentiated from classes 4 and 5. In some sense class 2 is "in between" these two classes for a lot of the statistics so it's hard to parse it from either class 4 or 5. We could expect a lot of misclassification between these two classes.

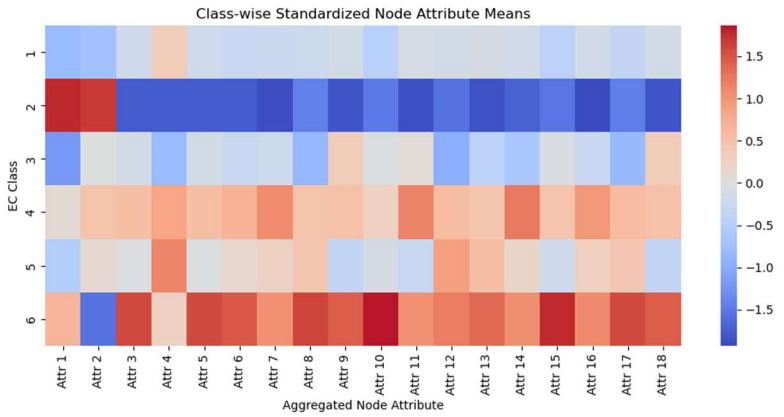

Figure 2. Heatmap of Node Attributes

To further explore differences at the node level, we examined the class wise standardized means of the aggregated node attributes (Figure 2). After standardizing, clear patterns emerge across enzyme classes, with certain classes consistently exhibiting higher or lower values across multiple attributes. For example class 2 tends to show uniformly lower values across many features, while class 6 exhibits consistently higher values, indicating strong separation in the underlying attribute space. Other classes, such as 3, 4 and 5 display more mixed patterns, suggesting partial overlap and more complex relationships. Importantly no single attribute cleanly separates all classes, rather each class is characterized by a combination of features. This indicates that the predictive signal is distributed across multiple node attributes and likely involves interactions between them.

4. Classical Machine Learning Results

To evaluate the predictive power of the aggregated graph features, we applied several classical machine learning techniques including Logistic Regression, Support Vector Machines (SVM), Random Forest, and XGBoost. All models were evaluated using 6-fold stratified cross-validation, ensuring consistent class distributions across folds. Performance was primarily assessed using macro F1 score, which weights each enzyme class equally and highlights differences in class specific performance.

Table 1. Comparison of Model Performance

Model	Accuracy	Macro F1	Precision	Recall
Logistic Regression	0.4633	0.4495	0.4527	0.4635
SVM (RBF Kernel)	0.5267	0.5024	0.5192	0.5262
Random Forest	0.6617	0.6598	0.6749	0.6614
XGBoost	0.6367	0.6338	0.6416	0.6364

The results in table 1 shows a clear separation of performance between model types. Random Forest achieved the best overall performance across all metrics, with a macro F1 of 0.6598, followed by XGBoost at 0.6338. Both tree based models consistently outperformed Logistic Regression and SVM which achieved substantially lower scores.

This gap suggests that the relationship between the aggregated graph features and enzyme class are likely nonlinear and involves feature interactions that are better captured by tree based models. In contrast, Logistic Regression assumes a linear decision boundary which struggles to capture this structure. Although SVM uses a nonlinear RBF kernel, its performance remains limited, indicating that the feature space may not be easily separable using distance-based methods alone.

Overall these results indicate that the aggregated node attribute features contain meaningful predictive signals, but that these signals are best exploited by flexible, interaction-based models such as Random Forest and XGBoost

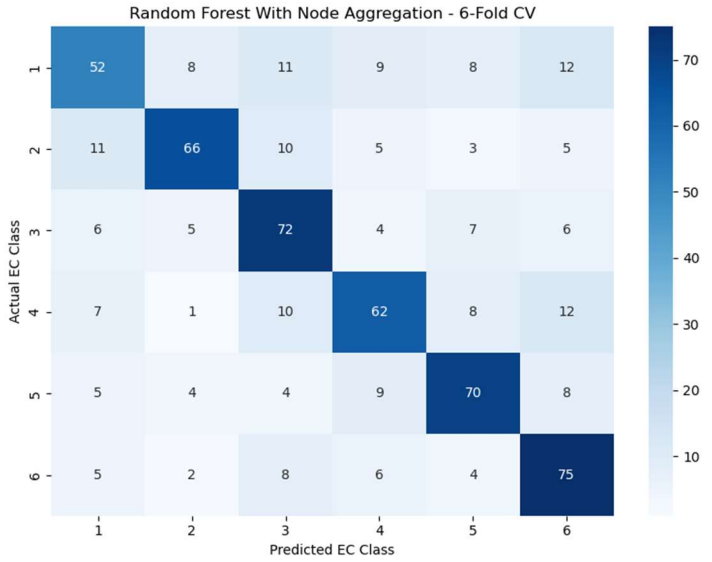

Figure 3. Random Forest Confusion Matrix

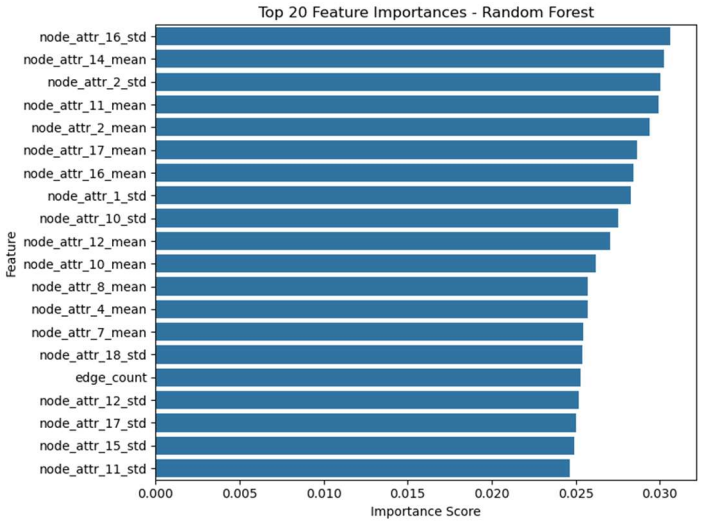

Figure 4. Random Forest Feature Importance

The confusion matrix for the Random Forest model shows that most enzyme classes are correctly identified, with strong performance on classes 3,5 and 6. Misclassifications tend to occur between neighboring or similar classes, suggesting some overlap in feature distributions rather than complete model failure.

The feature importance analysis further supports these findings, with aggregated node attribute statistics (means/standard deviations) dominating the top predictors. Structural graph features such as radius and diameter appear less influential, indicating node level information contributes more strongly to classification performance than global graph structure.

These findings are consistent with the EDA results, where class specific patterns in node attributes were observed, reinforcing that node level information drives the predictive performance of the models.

5. GCN Implementation

The first model we will discuss is the GCN. The model was implemented using the PyTorch package which allows us to parse each edge into a host of 600 graphs and pass them

through the package's graph based neural network models. We decided to write the model architecture in plain PyTorch instead of using a higher level package like PyTorch Geometric or DGL in order to have closer control and transparency of the neural network architecture.

I will be skipping the technical details of the preprocessing for the python models as it has a lot of specific quirks with how TU has set up the dataset. Generally, we build the graphs by filling in empty adjacency matrices for each graph based on the edge list and then filling in a node attribute matrix. The node attribute matrix has the 18 continuous predictors described earlier along with 3 more one hot encoded columns. Notably, we scale the continuous predictors but ensure not to scale the one hot encoded node labels. This allows us to have more stable predictions in the model so the scale of certain factors doesn't affect the weights too much. Notably we ensure to normalize the train and test sets separately in order to prevent data leakage. Finally, since the graphs are of varying sizes and we need to pass them as a tensor so we use padding. We identify the largest graph in the batch and make sure every graph is packed into the top left padded with 0s on the bottom and right in order to make sure the dimension of every graph is the same.

The GCN we have implemented is based on the standard normalized graph convolution formulation. Much like any convolutional network, the model repeatedly updates each node representation by combining transformed information from the node as well as its neighbors. The formula we used is $H^{(l+1)} = \sigma\left(\hat{D}^{-\frac{1}{2}}\hat{A}\hat{D}^{-\frac{1}{2}}H^{(l)}W^{(l)}\right)$ where $\hat{A} = A + I$ adds self loops to the adjacency matrix, $\hat{D}$ is the degree matrix of A hat, $H^{(l)}$ is the node feature matrix for layer l, $W^{(l)}$ is the weight matrix for layer l, and σ is the ReLU function.

This formula functions similarly to convolutions in a normal CNN, but defined using matrices. $\tilde{A} = \hat{D}^{-\frac{1}{2}}\hat{A}\hat{D}^{-\frac{1}{2}}$ is the normalized adjacency matrix which communicates the graph structure throughout the network, and it does not actually change. In the forward pass a node receives information from its neighbors, the preactivation state is considered to be $Z^{(l)} = \tilde{A}H^{(l)}W^{(l)}$. Taking the gradient of the loss with respect to this pre-activation we can recalculate the weights with $\frac{\partial L}{\partial W^{(l)}} = \left(H^{(l)}\right)^T \tilde{A}^T \frac{\partial L}{\partial Z^{(l)}}$ during the backpropagation. σ wraps the entire convolution operation used here in order to introduce non linearity into the model. The loss function we use here is a standard cross-entropy loss function. The implementation of the convolutional formula can be found in gcn_enzymes.py in the GCNLayer class' forward function which is the forward propagation for the convolutional layers.

The actual neural network architecture is relatively straightforward. There are 4 convolutional layers with a dropout layer then multi-stage pooling after each layer (with both mean and max pooling). The pooled representations (of which there are 8 since there are 2 for each convolutional layer) are stored in a list and passed through a readout layer with ReLU, followed by another dropout and then the final classification layer. The reason we have such deep convolutions and multiple pooling layers after each convolution is in order to be able to capture motifs on any scale. As the convolutions grow, we start focusing into smaller and smaller local areas within a protein, but as we have shown, global properties also matter. After all, the traditional machine learning methods performed much better than random guessing. This is why we pool after every convolution, even the first, and then concatenate them all for the final readout layer. This philosophy is also why it's a multi-stage pooling instead of just mean or max pooling. Mean pooling gives us the average behavior on each scale, but max pooling also allows us to look at particularly active nodes in any area of a graph. The final classifier does not depend solely on the deepest node embedding nor on the shallowest global graph properties, but a mix in

order to capture the global fold structure of proteins along with residue activities in small active sites.

A couple things of note in the implementation is firstly that we are using Adam as an optimizer. In addition, we use a stratified sample for the 6-fold cross validation. The choice of 6 folds for the classification is because it's quite close to a standard 80:20 split (83:17) and it's more interpretable in the context of this data set (500 training observations and 100 test). When running the actual model, the model stops itself when it's running the epochs if the loss goes up so it doesn't fully run for every epoch if the loss increases. The loss used here during training is not the test loss, the model is trained on a subtraining set and has its own validation set. This allows us to do hyperparameter tuning later without tuning specifically to one test fold. For hyperparameter tuning we used Optuna.

6. GAT Implementation

The GAT has the same overall workflow as the GCN but there's a key difference in how the graph's structure is communicated across the network. In a GCN the normalized adjacency matrix $\tilde{A} = \hat{D}^{-\frac{1}{2}}\hat{A}\hat{D}^{-\frac{1}{2}}$ is used and doesn't change. GATs fix this by using an attention mechanism which makes the network evaluate what neighbors are important for each node. This is particularly useful for proteins since in a protein graph, one local structural relation might be strongly associated with enzymatic function while another might be comparatively generic background context.

Since attention works dynamically between pairs of nodes, the GAT formula is written from the perspective of a single node rather than the whole graph matrix at once. The core formula for node i in a GAT is $h_i^{(l+1)} = \sigma\left(\sum_{j\in\mathcal{N}(i)\cup\{i\}} \alpha_{i,j}^{(l)} W^{(l)} h_j^{(l)}\right)$. Where *i* is the current node and *j* is the current node. Similarly to the GCN, W and h are the weight and features, however since we are looking at a single node, h is a vector of the current features instead of a matrix. The key factor here is $\alpha_{i,j}^{(l)}$ which is the attention coefficient for the current node and its neighbor. Since this is done for each node, two nodes may have differing weights than each other for the same connection. $\mathcal{N}(i) \cup \{i\}$ means the attention mechanism iterates over every node i is connected to including itself so it doesn't forget its own information.

The underlying attention mechanism is additive. After node features are projected into a learned latent space, the model forms a source-side attention score and a destination-side attention score for each head. These are added and passed through a LeakyReLU $e_{i,j} = \text{LeakyReLU}\left(\mathbf{a}^T[Wh_i \parallel Wh_j]\right)$. Node i calculates this for all its neighbors, so to make these usable we pass them through a Softmax function so all the attention weights for node i add up to 1 $\alpha_{i,j} = \frac{\exp(e_{i,j})}{\sum_{k\in\mathcal{N}(i)\cup\{i\}} \exp(e_{i,k})}$. The result is a distribution of attention weights for each node over its neighborhood. Neighbor embeddings are then combined by a weighted sum rather than by a fixed average. This mechanism lets the model decide that some neighbors should contribute strongly while others should contribute only weakly.

The GAT here also uses multi-head attention as is standard. Rather than learning a single set of neighborhood weights, each attention layer learns several heads in parallel. Each head computes its own attention distribution and is concatenated to form the node representation going forward. This stables the optimization relative to relying on one attention calculation alone.

In our technical implementation of the GAT, we share the GCN python files' dataset parsing and formatting to ensure it's being interpreted the same. The overall structure of them is

the same as well, four propagation layers, pooling after every layer, a dense readout and then the final classifier. The main difference is that the propagation layers are now attention layers and the nonlinearity is ELU rather than ReLU. We wanted to keep the structure the same in order to identify the differences in GCN and GAT performance without it being muddled by differences in the overall structure of the neural network. As such the GAT also used Adam as an optimizer and optuna as the hyperparameter sweep, though it has an extra hyperparameter in the number of attention heads.

7. Neural Network Results

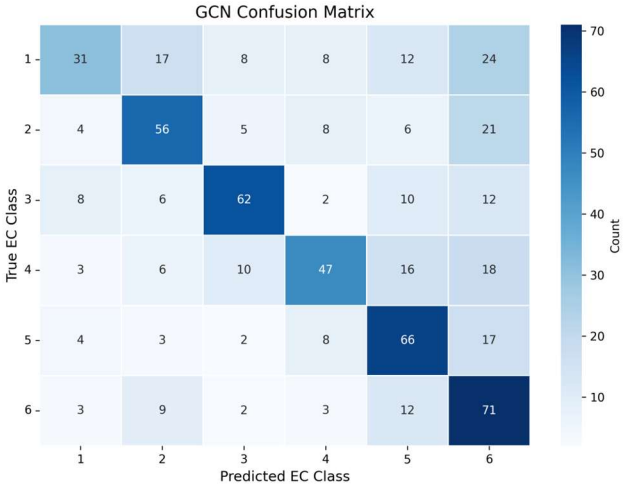

Figure 5. GCN Confusion Matrix

The GCN's chosen hyperparameters were as follows - hidden_dim: 128, dropout: 0.4, learning_rate: 0.002, batch_size: 32, and weight_decay: 0.0005. The mean test macro F1 for the model is 0.55 with a precision of 0.58 and recall of 0.56. As we can see, it's significantly better than the random forest model as we predicted. However, we still run into a good amount of issues. Class 1 in particular is concerning as it has the lowest F1 score of 0.41 and a recall of only 0.31 indicating that there's a lot of false negatives when it comes to classifying EC class 1. We can look into what classes are specifically causing this problem with the confusion matrix for the GCN in figure 5.

As we can see, class 1 has the lowest correct classifications. Interestingly, the misclassifications are split relatively evenly across the other classes, meaning class 1 might have very few unique features at all compared to any other class. Interestingly, every class was misclassified as class 6 the most. It seems that class 6 has most varying protein structures within the class since it has so much overlap in the predictions for other classes. Interestingly, class 6 has a recall of 0.71 meaning we were able to find many actual positives with few false negatives but the precision of 0.44 is quite low showing the false positive problem. It seems class 6 seems to cannibalize a lot of predictions from other classes, and it causes problems in predictive performance.

The chosen hyperparameters for the GAT are - hidden_dim: 16, heads: 8, dropout: 0.25, learning_rate: 0.0005, batch_size: 8, weight_decay: 1e-05. We can see some key differences in the hyperparameter choices here, the hidden_dim looks much smaller, but due to the heads it actually ends up being the same "size" as the GCN (since 16 * 8 = 128). The low learning rate and batch size are a symptom of the attention mechanism being particularly sensitive compared to the convolution tools, meaning this model chose hyperparameters that treaded slowly. Notably the GAT also has a lower dropout and weight decay. This is because the attention

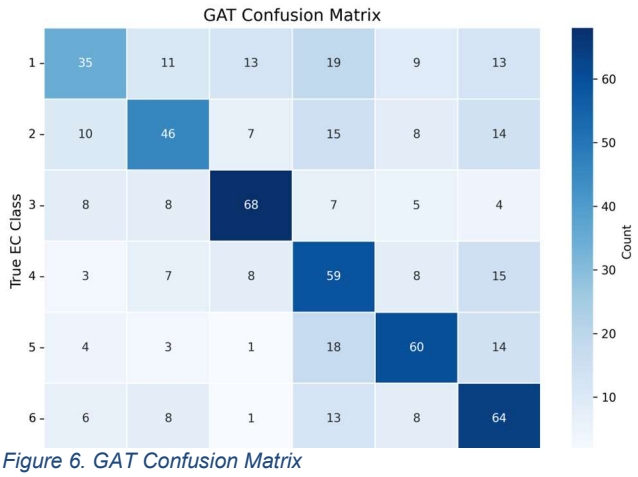

Figure 6. GAT Confusion Matrix

mechanism acts as a form of regularization so it naturally filters out a lot of the things that usually cause overfitting when compared to GCNs. This means the dropout and weight decay are less important. Overall the GAT is less broad and looks deeper into the structure of the training set. The GAT had a mean test macro F1 of 0.55, with a precision of 0.56 and recall of 0.55. These numbers are quite similar to those of the GCN, but there are differences within the classes that we can see.

Figure 6 shows the confusion matrix for the GAT and we notice immediately that once again class 1 has issues with being correctly classified. The F1 score is once again low at 0.42 with problems in the recall being only 0.35. Looking at where specifically the model is confusing class 1 paints a different picture, class 6 was a problem for cannibalizing the predictions of every other class, but here class 1 has more confusion with class 4. In general, the classes seem to misclassify amongst each other relatively evenly. Class 6 has a precision much higher of 0.52, meaning the false positive problem isn't particularly high for class 6. Class 4 instead seems to have the most false positives with a precision of 0.45. The overall performance of these models seems to be similar by the numbers, but it's clear the specific structures each model is catching on to are different.

In addition, for each GNN we found the validation and train accuracy and loss across the epochs so we could track how the models were trained. Most of the results were intuitive, as the models go deeper into the epochs the accuracy raises then plateaus and the loss lowers than plateaus. These graphs are contained in Figure 7, where the top row has the GCN graphs, the bottom row has the GAT graphs, the left column has accuracy curves, and the right has loss curves. The red is validation and the blue is for training

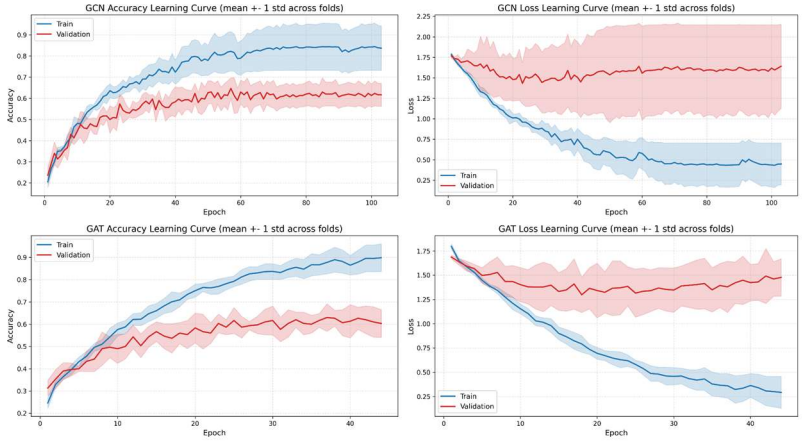

Figure 7. GAT/GCN Loss/Accuracy Across Epochs

One thing of note is the GAT loss curve (bottom right). The GCN training loss curve plateaus around 40-50, along with the validation loss. However, the GAT's training loss seems to continuously decrease despite the validation loss plateauing relatively early. In fact, the validation loss increases a little in the end. These are clear signs of overfitting in the GAT. As mentioned earlier, the GAT should control this on its own, however this is clearly not the case. The GAT having node level attention weights compared to the GCN's graph wide weights means that even though they have the same number of propagation layers the GAT seems to overfit. It seems when handling GATs, we need to account for its built in smaller fits and use shallower architectures compared to GCNs.

Another thing worth looking into is that the GCN seems to have a much higher standard deviation for the validation loss. We would think this to be the other way around due to overfitting in the GAT, however it seems the granularity of node-wise attention weights makes the model less uncertain in predictions. As a whole GNNs have many differences between them, but they have shared advantages compared to machine learning methods.

8. Comparisons and Results

As we can see, our assumptions at the start of the paper were false. The classic machine learning methods like Random Forest and XGBoost ended up outperforming our implementations of the GNNs. The F1-score of each is above 0.6, much more than that of the deep learning methods. It seems that although GNNs are well suited to protein classification in theory due to the aggregation of local structure and biochemical node attributes, the current implementations are performing much worse.

The GNNs seem to have a harder task and not enough data to solve it. The tree-based models are trained on graph level summaries like global network properties and aggregation of the node attributes. These inputs compress the protein into a representation that is compact and includes all necessary information in each protein. However, the GCN and GAT receive only the node level features and adjacency structure and has to learn both the local representations and the graph level summaries at the same time with only 600 protein graphs. With such little data we ended up finding that the GNNs weren't as competitive as they should be in theory.

A second issue is that the implemented architecture may not actually preserve the signals we wanted it to. Right now the GNN is treating each protein as undirected, unweighted graphs with node attributes and node labels, but it lacks deeper geometric or biochemical edge information like residue distances, interaction types, or angular structure. The current GCN also uses repeated neighborhood averaging by nature in its formulation which can smooth away subtle local motifs rather than expanding them. Both the GCN and GAT reduce node embeddings to graph predictions through global mean/max pooling, which can also dilute microscopic structures. Overall, the current GNNs are not failing due to being deep learning models but because the present architecture doesn't have good architecture to infer global summaries while the tree-based methods are directly given strong graph level descriptors.

However, the GNNs do in fact outperform machine learning methods like logistic regression and SVMs. This is most likely because GNNs are inherently more flexible and don't fall into inflexible linear assumptions. We see this even when comparing the classical machine learning methods with each other, we need particularly flexible methods to capture protein networks much like we suggested in the introduction to our paper.

9. Conclusions and Future Work

In this project, we managed to successfully apply both traditional machine learning and advanced deep learning methodologies to classify enzyme proteins into their respective EC numbers based on their graphical representations. By modeling proteins as networks, we were able to show that both global structure as well as local residue structures contain real predictive power for enzyme functions. Every model has significantly outperformed the random guess probability of 16.67%. However, we also found shortcomings in some of the models.

Initially, we hypothesized that GNNs would vastly outperform classical machine learning methods, however we found the opposite. The tree-based methods' superior performances highlighted that by preprocessing the data into valuable aggregates, it made it much easier for machine learning methods to capitalize on these features. In contrast, the GNNs had to learn both microscopic and macroscopic methods from scratch. With only 600 graphs the GNNs did not have enough data to capably learn both representations as we had hopes. In addition, there were clear issues with our architectural choices like deep pooling and over-smoothing. Ultimately GNNs performed better than rigid machine learning methods like Logistic Regresion and SVMs but couldn't outperform the tree-based methods with our current implementation

This leaves a lot of room for future work. As mentioned earlier we can implement things like shallower GNN architectures or have richer data about these proteins. In addition, in the realm of GNNs there are some specific model choices that could have led to a better outcome for the deep learning models. We could have directly added the aggregate graph level features to the GNN model. We could also use geometric GNNs like SchNet or EGNNs that can match protein structure closer than plain adjacency-based models like we used. A key error was in the pooling method, replacing these simple mean/max pooling layers with things like attention pooling, Set2Set or hierarchical pooling can preserve more local motifs than mean pooling especially. Ideally, we would also be able to generate a modern RCSB PDB protein-based training and test set so we can see how it performs after the introduction of EC class 7 in 2018. This may clear up some issues that the GNNs or even the machine learning methods had. There's clearly a lot of work to be done on possible issues in this project. While it's a good exploration of the beginnings of GNNs, how they work, and how they're implemented there are many advancements that could make these neural networks very well suited to the task even with things as simple as architectural decisions.

All the code for this analysis can be found in the GitHub which is also located at the start of this report.

10. Team Contributions

Saketh led the core development for the deep learning part of this project. Specifically, he designed and implemented the architectures for the GCN and GAT architectures and implemented the 6-fold cross validation for each along with the hyperparameter sweep. He also did the EDA for the global network statistics across the EC classes. Based on this and the model results he generated the confusion matrices for each of the 3 models as well as the feature importance for the random forest. Finally, he authored sections 1-3 and 5-8 of this report and presented the mid-semester project review.

Shruthi built the evaluation infrastructure across all three classifier pipelines (GCN, GAT, Random Forest). For graph neural networks, she instrumented the training loop to capture per-epoch metrics across six-fold cross-validation, implementing right-padding alignment to handle variable-length fold histories from early stopping and computing mean ± std learning curves. For Random Forest, she leveraged scikit-learn's warm_start to grow forests incrementally from 10 to 300 estimators, using out-of-bag accuracy as an unbiased generalization proxy. Confusion matrices aggregate predictions across all held-out folds, ensuring the full 600-graph dataset is represented in error analysis across six enzyme commission categories.

Ryan developed the classical machine learning models used in the project, including Logistic Regression, Support Vector Machines (SVM), Random Forest, and XGBoost. He integrated the aggregated node attribute features into the modeling pipeline and implemented 6-fold stratified cross-validation for consistent evaluation across all models. He conducted comparative performance analysis using accuracy, macro F1, precision, and recall, and generated the model comparison table presented in Section 4. Additionally, he created the confusion matrix and feature importance visualizations for the Random Forest model and performed node-level exploratory data analysis, including the class-wise standardized attribute heatmap. He authored Section 4 (Classical Machine Learning Results) and contributed to interpreting how feature structure influences model performance.